

Documentation Technique — Bibliothèque en ligne

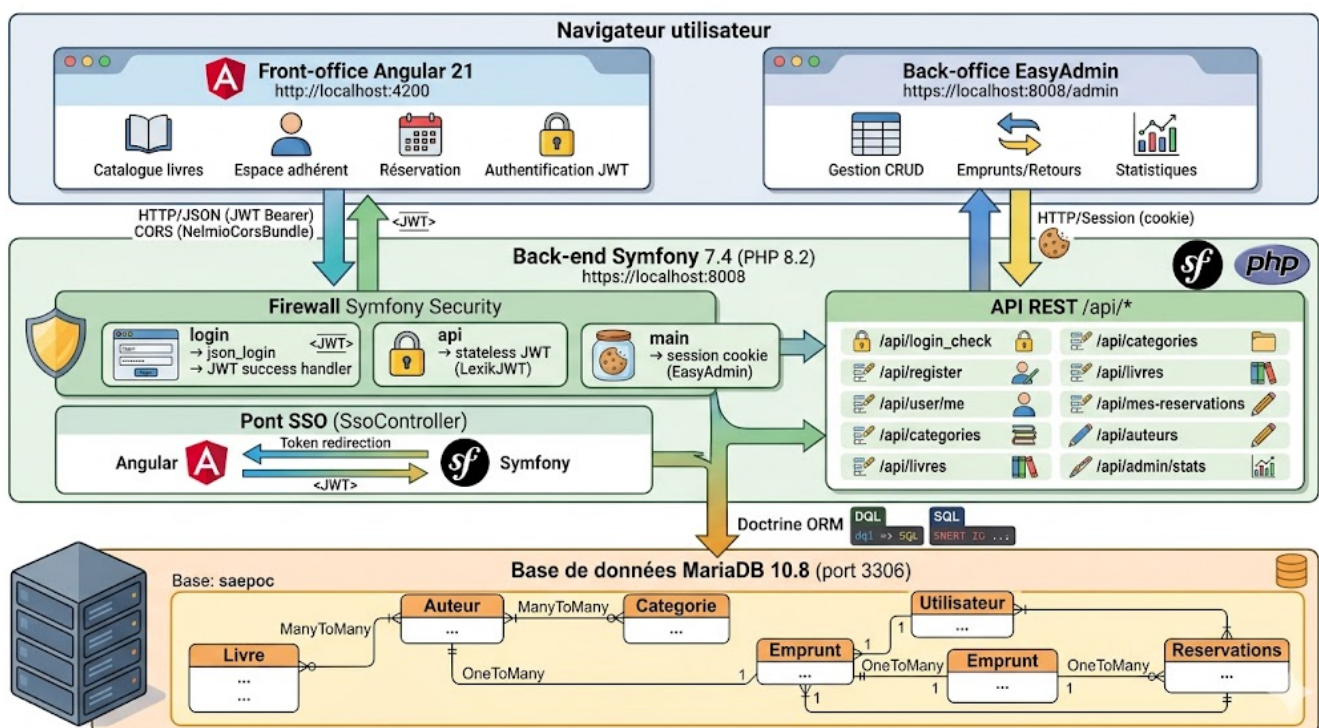
SAE S6.A.01 — **Symfony / Angular** BUT Informatique 3e année — 2025/2026

Table des matières

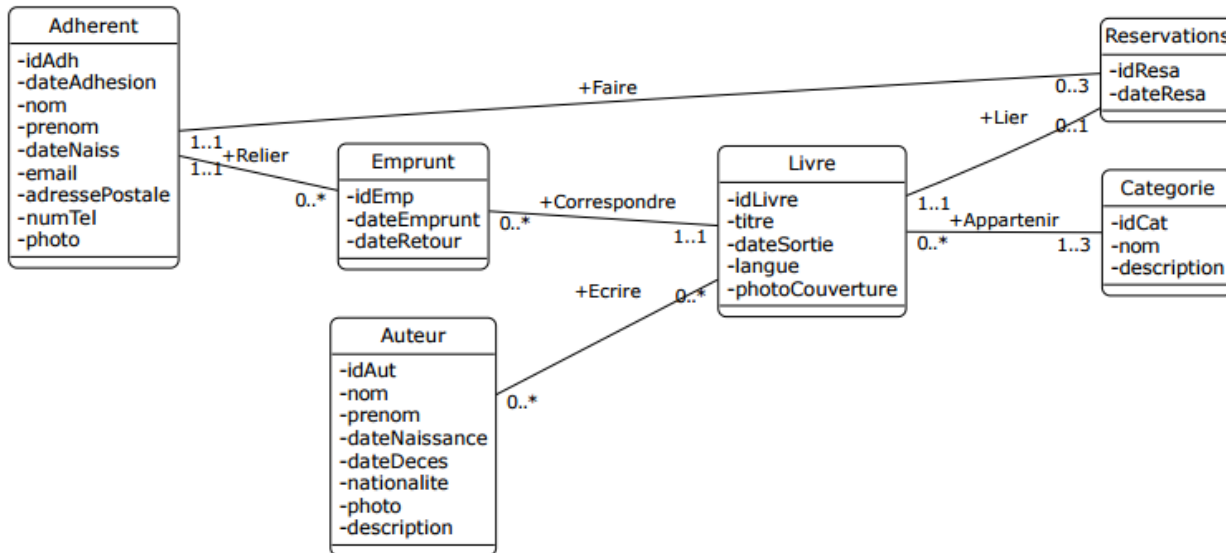
1. [Architecture du projet](#)
2. [Choix techniques](#)
3. [Guide d'installation](#)
4. [Difficultés rencontrées et solutions apportées](#)
5. [Répartition du travail](#)

1. Architecture du projet

1.1 Schéma global



1.2 Diagramme de classes



1.3 Structure des dossiers

```

poc_articles/                               Symfony (back-end + API + back-office)
├── src/
│   ├── Controller/
│   │   ├── Api/           Contrôleurs REST (JSON)
│   │   ├── Admin/        Contrôleurs EasyAdmin (CRUD)
│   │   ├── HomeController  Redirection / vers /admin
│   │   ├── SecurityController  Login / Logout session
│   │   └── SsoController   Pont JWT ↔ session
│   ├── Entity/           Entités Doctrine
│   ├── Repository/       Requêtes personnalisées
│   ├── Validator/        Règles métier (EmpruntRules, ReservationRules)
│   ├── Security/         AdminAuthenticator
│   └── DataFixtures/     Données de test
├── config/
│   └── packages/
│       ├── security.yaml  Firewalls + access control
│       ├── lexik_jwt_authentication.yaml
│       └── nelmio_cors.yaml
└── migrations/           Migrations Doctrine

front_articles/                          Angular (front-office)
├── src/app/
│   ├── components/       Composants (home, livres-list, livre-detail,
│   │                   auteurs-list, auteur-detail, categories-list,
│   │                   profil, login, admin-dashboard)
│   ├── services/         ApiService, AuthService
│   ├── guards/           authGuard, adminGuard
│   ├── interceptors/     authInterceptor (Bearer token + 401)
│   ├── models/           Interfaces TypeScript
│   └── environments/     URLs d'API

mariadb-fp/                      MariaDB 10.8 portable (Windows)
  
```

2. Choix techniques

2.1 Stack et versions

Composant	Choix	Version
Back-end	Symfony	7.4
Back-office	EasyAdminBundle	4.x
Authentification API	LexikJWTAuthenticationBundle	3.x
CORS	NelmioCorsBundle	2.x
PDF	dompdf	3.x
ORM	Doctrine	3.x
Front-end	Angular	21
CSS	Bootstrap 5 + Bootswatch	5.3
Base de données	MariaDB	10.8
PHP	—	8.4
Node.js	—	>= 22

2.2 Authentification double mécanisme

L'application expose deux interfaces distinctes qui n'utilisent pas le même mécanisme d'authentification.

API REST (Angular) : authentification stateless par JWT. Le client envoie `Authorization: Bearer <token>` à chaque requête. Le token est stocké dans le `localStorage` du navigateur. L'interceptor Angular l'injecte automatiquement et redirige vers `/login` en cas de 401.

Back-office EasyAdmin : authentification par session PHP cookie classique, nécessaire car EasyAdmin repose sur le firewall `main` de Symfony qui est stateful.

Pour permettre à un utilisateur connecté sur Angular d'accéder à EasyAdmin sans se reconnecter (et inversement), un pont SSO a été mis en place via `SsoController` :

- Angular → EasyAdmin : l'Angular passe son JWT dans l'URL (`/sso/to-symfony?token=<JWT>`), Symfony decode le token, retrouve l'utilisateur et ouvre une session.
- EasyAdmin → Angular : Symfony génère un nouveau JWT et redirige vers le front avec `?token=<JWT>` dans l'URL.

2.3 Hiérarchie des rôles

```
ROLE_ADMIN
├── ROLE_BIBLIO
│   └── ROLE_USER
```

- **ROLE_USER** : adhérent — peut consulter le catalogue, réserver, voir son espace.
- **ROLE_BIBLIO** : bibliothécaire — accès à EasyAdmin, gestion emprunts/réservations/catalogue.
- **ROLE_ADMIN** : responsable — hérite de tout + accès au dashboard de statistiques Angular.

2.4 Structure des entités

Livre est l'entité centrale. Elle est liée en ManyToMany à **Auteur** et **Categorie** (un livre peut avoir plusieurs auteurs et inversement). Les relations **Emprunt** et **Reservations** sont OneToMany côté **Livre** et côté **Utilisateur**.

Le choix du ManyToMany sans entité pivot explicite est justifié par l'absence de données supplémentaires sur la liaison (pas de rôle d'auteur, pas de position). Doctrine gère la table de jointure automatiquement.

Emprunt.dateRetour est nullable : un emprunt sans date de retour est un emprunt en cours. C'est ce champ qui sert de discriminant dans toutes les requêtes (livres disponibles, retards, statistiques).

La durée de prêt maximale de 15 jours est calculée à la volée dans **Emprunt::getIsEnRetard()** via un clone de **dateEmprunt** (pas stockée en base).

2.5 Règles métier via contraintes de validation Symfony

Plutôt que d'éparpiller les contrôles dans les contrôleurs, les règles métier sont centralisées dans des classes **Constraint** / **ConstraintValidator** personnalisées annotées directement sur les entités :

- **#[EmpruntRules]** sur **Emprunt** : vérifie que l'adhérent n'a pas déjà 5 emprunts en cours et que le livre est disponible.
- **#[ReservationRules]** sur **Reservations** : vérifie la limite de 3 réservations par adhérent et que le livre n'est pas déjà réservé ou emprunté.

Ces contraintes sont déclenchées aussi bien lors de la création via l'API que via EasyAdmin, garantissant une cohérence des règles quel que soit le point d'entrée.

2.6 Recherche et pagination

La recherche de livres est implémentée dans **LivreRepository::findBySearchCriteria()** via le QueryBuilder Doctrine. Les trois filtres (titre, auteur, catégorie) sont cumulables et utilisent des **LIKE** case-insensitive. La pagination utilise la classe **Doctrine\ORM\Tools\Pagination\Paginator** (gère correctement les jointures ManyToMany). La réponse API retourne un objet **{ data, totalPages, currentPage }** consommé directement par Angular.

Paramètres : 9 livres par page (configurable dans le repository).

2.7 Groupes de sérialisation

Pour éviter les boucles de référence et contrôler précisément les données exposées par l'API, le composant sérialiseur Symfony est utilisé avec des groupes :

- **livre:read** : données du livre + auteurs + catégories (sans les emprunts).
- **user:read** : données de l'utilisateur + ses emprunts + ses réservations (avec résumé livre).
- **auteur:read** : données de l'auteur seul.
- **categorie:read** : données de la catégorie seule.

2.8 Back-office EasyAdmin

EasyAdmin est choisi pour sa capacité à générer rapidement une interface d'administration complète à partir des entités Doctrine, sans écrire de templates HTML. Les personnalisations notables :

- **EmpruntCrudController** : action personnalisée "Enregistrer le retour" (met à jour **dateRetour**) ; indicateur visuel retard (badge HTML rouge/vert) ; export fiche PDF via dompdf.
- **ReservationsCrudController** : action "Transformer en emprunt" qui crée un **Emprunt** depuis la **Reservation** puis supprime la réservation, avec validation des règles métier avant persistance.
- **UtilisateurCrudController** : hachage du mot de passe à la création et à la mise à jour (détection si déjà haché avec le préfixe **\$argon**).
- **Dashboard** : affichage de KPI (nb livres, adhérents, catégories, emprunts en cours) via Doctrine **count()**.

2.9 Front-office Angular

Architecture standalone (Angular 21, pas de NgModule). Chaque composant déclare ses imports directement.

Gestion d'état : **AuthService** utilise les **signals** Angular pour que les composants réagissent automatiquement aux changements de connexion (affichage conditionnel de la barre de navigation notamment).

Routage : protégé par deux guards fonctionnels (**authGuard**, **adminGuard**). Les routes **/profil** et **/admin/dashboard** sont inaccessibles sans être connecté ou sans le bon rôle.

3. Guide d'installation

Prérequis

Installer les logiciels suivants :

Outil	Version minimale	Lien
PHP	8.4	https://windows.php.net/download/
Composer	2.x	https://getcomposer.org/download/
Symfony CLI	dernière	https://symfony.com/download
Node.js	22 LTS	https://nodejs.org/
Git	—	https://git-scm.com/

Si vous n'avez pas installer : PHP

Téléchargez et installez PHP 8.4 via le lien fourni

- Décompressez le zip et placez le dans C:\Programme Files \PHP
- Ajouter le chemin d'accès dans votre PATH
- Vérifiez l'installation en ouvrant une invite de commande et en tapant : `php -v`

- Il est possible que votre ordinateur n'ait pas les DLL MS C++ redistribuables installées.
- Si c'est le cas, téléchargez-les et installez-les depuis le site officiel de Microsoft.

Il faudra aussi modifier le php.ini :

- Si vous ne trouvez pas de php.ini dans le répertoire php, renommez php.ini-development en php.ini.
- Il faudra par la suite le modifier afin d'activer un certain nombre d'extensions :
 - Pour Windows, il suffit de décommenter les lignes suivantes :

```
; À activer dans php.ini
extension_dir = "ext"

extension=curl
extension=fileinfo
extension=intl
extension=mbstring
extension=openssl
extension=sodium      ; calcul de clé de hachage

extension=mysqli      ; MySQLi (connexion MySQL native)
extension=pdo_mysql    ; MySQL avec PDO
extension=pdo_sqlite   ; SQLite avec PDO
```

Pour symfony :

- Télécharger le binaire sur votre PC via le lien fourni.
- En fonction de votre PC, vous avez le choix de télécharger une version 386 ou amd64
- Le zip contient l'exécutable symfony.exe, copier le dans le répertoire PHP (qui est déjà dans le PATH)
- Vérifiez l'installation avec la commande : symfony check:requirements.
- Vérifiez la version avec la commande : symfony version.
- Notre version : Symfony CLI 5.16.

Pour node et npm :

- Téléchargez et installez Node.js 22 depuis le site officiel. npm est inclus avec Node.js

MariaDB est fourni dans le dépôt (dossier [mariadb-fp/](#)), aucune installation séparée n'est requise.

Vérifier une nouvelles fois les installations :

```
php -v
composer -V
symfony version
node -v
npm -v
```

Étape 1 — Récupérer le projet

```
git clone https://github.com/AchrafAmeri/S6.A.01-Symfony-Angular-Biblio.git
cd S6.A.01-Symfony-Angular-Biblio
```

Étape 2 — Démarrer MariaDB

Double-cliquer sur `mariadb-fp/start-server.bat` (ou l'exécuter depuis un terminal).

Le serveur MariaDB écoute sur le port **3306**.

Vérifier que la base `saepoc` existe (accessible via Adminer sur `http://localhost:8008/adminer` une fois Symfony lancé, ou via `mysql-client.bat`). Si elle n'existe pas, la créer :

```
CREATE DATABASE saepoc CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE saepoc
// Vérification
SHOW TABLES;
```

Étape 3 — Configurer le back-end Symfony

```
cd poc_articles
```

Copier le fichier d'environnement :

```
cp .env .env.local
```

Éditer `.env.local` et renseigner la connexion à la base de données :

```
DATABASE_URL="mysql://root:@127.0.0.1:3306/saepoc?serverVersion=10.8.3-
MariaDB&charset=utf8mb4"
```

Le mot de passe root de MariaDB fourni est vide par défaut dans la configuration locale.

Renseigner également l'URL du front Angular (pour le pont SSO et CORS) :

```
APP_ANGULAR_FRONT_URL=http://localhost:4200
```

Étape 4 — Installer les dépendances PHP

```
composer install
```

Étape 5 — Générer les clés JWT

```
php bin/console lexik:jwt:generate-keypair
```

Cela crée `config/jwt/private.pem` et `config/jwt/public.pem`.

Étape 6 — Exécuter les migrations

```
php bin/console doctrine:migrations:migrate --no-interaction
```

Étape 7 — Installer HTTPS (si ce n'est pas déjà fait)

```
symfony server:ca:install
```

Étape 8 — Démarrer le serveur Symfony

```
symfony server:start
```

Le back-end est accessible sur **`https://localhost:8008`**.

La Symfony CLI génère automatiquement un certificat TLS local. En cas d'avertissement navigateur, accepter l'exception de sécurité ou installer le certificat CA avec `symfony server:ca:install`.

Étape 9 — Configurer et démarrer le front Angular

Ouvrir un nouveau terminal :

```
cd front_articles  
npm install
```


Vérifier `src/environments/environment.ts` :

```
export const environment = {  
  production: false,  
  apiUrl: 'https://localhost:8008/api',  
  ssoUrl: 'https://localhost:8008/sso/to-symfony',  
  logoutUrl: 'https://localhost:8008/logout',  
  frontUrl: 'http://localhost:4200'  
};
```

Démarrer le serveur de développement :

```
npm start
```

Le front-office est accessible sur **http://localhost:4200**.

Accès à l'application

Interface	URL
Front-office Angular	http://localhost:4200
API Symfony	https://localhost:8008/api
Back-office EasyAdmin	https://localhost:8008/admin
Adminer (base de données)	https://localhost:8008/adminer

Compte administrateur par défaut (créé par les fixtures) :

Champ	Valeur
Email	admin@articles.fr
Mot de passe	admin
Rôle	ROLE_ADMIN

4. Difficultés rencontrées et solutions apportées

Nous avons rencontrés plusieurs difficultés rencontrées lors de ce projet pour la plupart d'entre nous :

- Mohammed : Difficulté sur l'architecture hybride (Back-Office HTML/Twig + API REST JSON) sans dupliquer les règles métier. Solution : centraliser la logique métier dans des services Symfony appelés par les contrôleurs Web et API, afin de garantir des comportements identiques (ex: disponibilité d'un livre). Difficulté aussi sur les relations Doctrine (Livre/Auteur/Catégorie/Emprunt), avec risque de requêtes N+1 et de suppressions non maîtrisées. Solution : optimiser les lectures avec des jointures (**JOIN**/fetch join via QueryBuilder) et configurer les relations avec prudence (éviter **cascade** :

["remove"] sur des relations partagées comme ManyToMany, privilégier la dissociation des liens et des règles explicites de suppression).

- Pierre-Louis : Difficulté sur la gestion des codes d'état HTTP dans l'API (éviter de renvoyer **200 OK** en cas d'erreur, ce qui induit le front en erreur). Solution : appliquer les codes standards selon le cas (**201 Created** après création, **400 Bad Request** pour des données invalides, **403 Forbidden** en cas d'accès refusé, **404 Not Found** si la ressource n'existe pas). Difficulté également sur la sécurisation stateless des routes API (sans session serveur). Solution : authentification par JWT ; le token est validé à chaque requête par le firewall API Symfony, ce qui permet d'authentifier l'adhérent sans session persistante côté serveur.
- David : Premières difficultés lors du paramétrage du projet sur sa machine personnelle (commande pas reconnu alors que tous les outils d'installer), et difficultés lors de l'intégration et la consommation de l'API REST dues à des notions encore fragiles sur la programmation asynchrone en TypeScript. La solution a été d'utiliser systématiquement **HttpClient** avec des **Observable** : chaque appel HTTP retourne un **Observable<T>** auquel le composant souscrit via **.subscribe()**, ce qui permet de traiter la réponse de manière asynchrone sans bloquer le thread principal.

5. Répartition du travail

Membre	Périmètre
Mohammed	Exigences back-office : interface EasyAdmin (Symfony), gestion CRUD, actions personnalisées (retour emprunt, transformation réservation → emprunt, export PDF), tableau de bord administrateur
Pierre-Louis	Exigences API REST : conception et implémentation de l'ensemble des endpoints Symfony (/api/*), authentification JWT, gestion des réservations côté API, règles de validation métier
David	Exigences front-office : application Angular (composants, services, guards, interceptor, routing), intégration de l'API, gestion de l'authentification côté client, pont SSO